

Node Similarity: K-Nearest Neighbors (KNN)

Neo4j Graph Data Science Library

Overview

- Computes distance for **all node pairs** using **node properties**
- Creates new relationships between each node and its **k nearest neighbors**
- Input: **homogeneous graph** (Any node label or relationships type information in the graph is ignored)
- Scales **quasi-linear** (not quadratic)

How It Works

1. Pick `k` random initial neighbors per node
2. Iteratively refine neighbor lists by comparing **neighbors-of-neighbors**
3. Stop when changes fall below `deltaThreshold` or reach `maxIterations`
4. Output new `(:Node)-[:SIMILAR {score}]->(:Node)` relationships

Based on **Wei Dong et al.** — *Efficient k-nearest neighbor graph construction for generic similarity measures*

Similarity — Scalar Numbers

Single numeric property:

$$\textit{similarity} = \frac{1}{1 + |a - b|}$$

Result in range (0, 1]

Similarity — List of Integers

Jaccard (default):

$$J(A, B) = \frac{|A \cap B|}{|A \cup B|}$$

Overlap coefficient:

$$O(A, B) = \frac{|A \cap B|}{\min(|A|, |B|)}$$

Both give scores in `[0, 1]` — no normalization needed

Similarity — List of Floats

Cosine (default):

$$\cos(A, B) = \frac{A \cdot B}{\|A\| \|B\|}$$

Pearson correlation:

$$r = \frac{\sum (x_i - \bar{x})(y_i - \bar{y})}{\sqrt{\sum (x_i - \bar{x})^2 \sum (y_i - \bar{y})^2}}$$

Euclidean similarity:

$$d = \sqrt{\sum (a_i - b_i)^2}, \quad score = \frac{1}{1 + d}$$

Cosine/Pearson range [-1, 1] normalized to [0, 1]

Multiple Properties

When using multiple properties, **similarity = mean of individual scores**

```
nodeProperties: [  
  {embedding: "EUCLIDEAN"},  
  'age',  
  {lotteryNumbers: "OVERLAP"}  
]
```

Each score is in $[0, 1]$, so the mean is also in $[0, 1]$

Node Properties Configuration

```
// Single property
nodeProperties: 'embedding'

// Map of property -> metric
nodeProperties: {
  embedding: 'COSINE',
  age: 'DEFAULT',
  lotteryNumbers: 'OVERLAP'
}

// List of mixed
nodeProperties: [
  {embedding: 'COSINE'},
  'age',
  {lotteryNumbers: 'OVERLAP'}
]
```


Available Metrics by Type

Type	Available Metrics
Scalar number	DEFAULT only
List of Integer	JACCARD (default), OVERLAP
List of Float	COSINE (default), EUCLIDEAN , PEARSON

Initial Neighbor Sampling

Uniform (default):

- Picks k random neighbors uniformly from all nodes
- Does **not** use graph topology

Random Walk:

- Depth-biased random walk from each node
- First k unique nodes visited become initial neighbors
- Uses graph topology — good when similar nodes are topologically close

Key Configuration Parameters

Parameter	Default	Description
<code>nodeProperties</code>	(required)	Properties & metrics for similarity
<code>topK</code>	10	Number of neighbors per node
<code>sampleRate</code>	0.5	Fraction of neighbors compared per iteration
<code>similarityCutoff</code>	0	Minimum similarity to include
<code>perturbationRate</code>	0	Probability to replace tied neighbors randomly
<code>initialSampler</code>	"uniform"	"uniform" or "randomWalk"

Stream Mode Syntax

```
CALL gds.knn.stream(  
  graphName: String,  
  configuration: Map  
)  
YIELD node1: Integer, node2: Integer, similarity: Float
```

Returns:

- `node1` , `node2` — Node IDs
- `similarity` — Computed similarity score in `[0, 1]`

Stats Mode

```
CALL gds.knn.stats('myGraph', {  
  topK: 1,  
  nodeProperties: ['age']  
})  
YIELD nodesCompared, similarityPairs,  
      ranIterations, didConverge,  
      nodePairsConsidered
```

Extra KNN-specific results:

- `ranIterations` — how many iterations ran
- `didConverge` — whether `deltaThreshold` was met
- `nodePairsConsidered` — number of similarity computations

Write Mode

```
CALL gds.knn.write('myGraph', {  
  writeRelationshipType: 'SIMILAR',  
  writeProperty: 'score',  
  topK: 1,  
  nodeProperties: ['age'],  
  randomSeed: 42,  
  concurrency: 1  
})  
YIELD nodesCompared, relationshipsWritten
```

Persists `(:Person)-[:SIMILAR {score}]->(:Person)`

Relationships are always **directed**

Deterministic Results

For reproducible output:

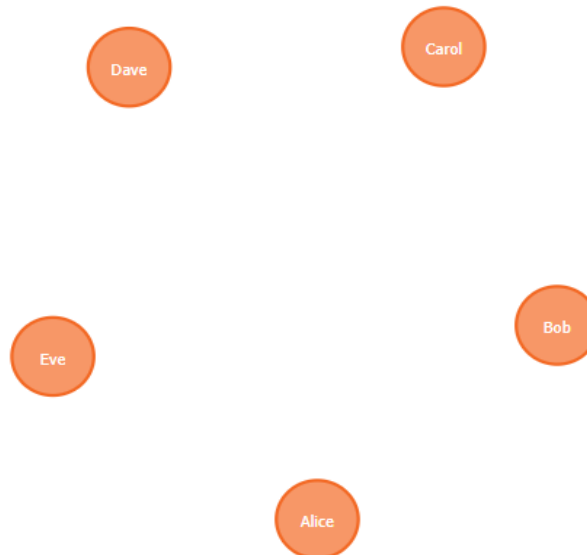
```
{  
  concurrency: 1,  
  randomSeed: 1337,  
  sampleRate: 1.0,  
  deltaThreshold: 0.0  
}
```

- `concurrency` must be 1
- `randomSeed` must be explicitly set

Example Graph — People

```
CREATE (alice:Person {name: 'Alice', age: 24, lotteryNumbers: [1, 3], embedding: [1.0, 3.0]})  
CREATE (bob:Person {name: 'Bob', age: 73, lotteryNumbers: [1, 2, 3], embedding: [2.1, 1.6]})  
CREATE (carol:Person {name: 'Carol', age: 24, lotteryNumbers: [3], embedding: [1.5, 3.1]})  
CREATE (dave:Person {name: 'Dave', age: 48, lotteryNumbers: [2, 4], embedding: [0.6, 0.2]})  
CREATE (eve:Person {name: 'Eve', age: 67, lotteryNumbers: [1, 5], embedding: [1.8, 2.7]});
```

All nodes are disconnected — KNN works on node properties alone.



The following statement will project the graph and store it in the graph catalog.

```
MATCH (p:Person)
RETURN gds.graph.project(
  'myGraph',
  p,
  null,
  {
    sourceNodeProperties: p { .age, .lotteryNumbers, .embedding },
    targetNodeProperties: {}
  }
)
```

Example Results — Single Property (age)

```
CALL gds.knn.stream('myGraph', {
  topK: 1,
  nodeProperties: ['age'],
  // The following parameters are set to produce a deterministic result
  randomSeed: 1337,
  concurrency: 1,
  sampleRate: 1.0,
  deltaThreshold: 0.0
})
YIELD node1, node2, similarity
RETURN gds.util.asNode(node1).name AS Person1, gds.util.asNode(node2).name AS Person2, similarity
ORDER BY similarity DESCENDING, Person1, Person2
```

Table 16. Results

Person1	Person2	similarity
"Alice"	"Carol"	1.0
"Carol"	"Alice"	1.0
"Bob"	"Eve"	0.14285714285714285
"Eve"	"Bob"	0.14285714285714285
"Dave"	"Eve"	0.05

Example — Multiple Properties

```
CALL gds.knn.stream('myGraph', {
  topK: 1,
  nodeProperties: [
    {embedding: "EUCLIDEAN"},
    'age',
    {lotteryNumbers: "OVERLAP"}
  ],
  // The following parameters are set to produce a deterministic result
  randomSeed: 1337,
  concurrency: 1,
  sampleRate: 1.0,
  deltaThreshold: 0.0
})
YIELD node1, node2, similarity
RETURN gds.util.asNode(node1).name AS Person1, gds.util.asNode(node2).name AS Person2, similarity
ORDER BY similarity DESCENDING, Person1, Person2
```

Example Results — Multiple Properties

Table 20. Results

Person1	Person2	similarity
"Alice"	"Carol"	0.8874315534
"Carol"	"Alice"	0.8874315534
"Bob"	"Carol"	0.4674429487
"Eve"	"Bob"	0.3700361866
"Dave"	"Bob"	0.2887113179

Accuracy vs. Performance Trade-off

sampleRate controls the trade-off:

Value	Accuracy	Memory	Speed
1.0	Highest	Highest	Slowest
0.5 (default)	Balanced	Moderate	Moderate
Low	Lower (may miss neighbors)	Lower	Faster

randomJoins — more attempts = better exploration, slower runtime

Convergence Control

```
{  
  maxIterations: 100,    // hard stop  
  deltaThreshold: 0.001  // early stop  
}
```

Algorithm stops when:

- Reaches `maxIterations`, or
- Fewer than `deltaThreshold` fraction of neighbors changed

Check `didConverge` in stats/mutate/write output.

Key Takeaways

- KNN = **property-based** similarity (no relationship traversal)
- Input: **homogeneous graph** — only node properties matter
- **Iterative refinement** via neighbors-of-neighbors
- Multiple metrics: **DEFAULT, JACCARD, OVERLAP, COSINE, EUCLIDEAN, PEARSON**
- **Quasi-linear** scaling — not $O(n^2)$
- Deterministic mode: `concurrency: 1` + `randomSeed`
- Four execution modes: `stream`, `stats`, `mutate`, `write`

References

- [KNN Documentation](#)
- [Efficient k-nearest neighbor graph construction](#) — Wei Dong et al.
- [Nearest neighbor graph](#) — Wikipedia
- [Product recommendations with kNN + FastRP](#)

